

UOS - Capstone

Week 3 - Research Questions: GAN and Diffusion Models

박상훈 ¹

2026학년도 2학기

¹Gwanak Lab Co., Ltd., Korea
E. sanghun.park@gwanaklab.co.kr

이번 주 진행 방식

Recap: 세 갈래의 생성 모델

GAN

Diffusion: 최소한의 배경

Research Questions — Diffusion

Research Questions — 세 모델 공통

정리

이번 주 진행 방식

이번 주에 하는 일

- 이번 시간은 **강의**가 아니라 **질문**이 중심입니다.
- 두 모델의 수식을 유도하지 않습니다. 대신 “**왜 그렇게 설계되었는가**”를 묻습니다.
- 오늘 다룰 질문은 **11개**입니다. (GAN 3 + Diffusion 6 + 공통 2)

질문 슬라이드 → 각자 생각 (2-3분) → 다음 슬라이드에서 답 확인

- 답 슬라이드의 내용은 **정답이 아니라 출발점**입니다. 각 슬라이드 아래의 **찾아볼 키워드**에서 스스로 더 조사하십시오.
- 일부 질문에는 **직접 확인해보기**가 붙어 있습니다. 말로만 이해하기 어려운 것들이니 코드로 확인해 보시기 바랍니다.

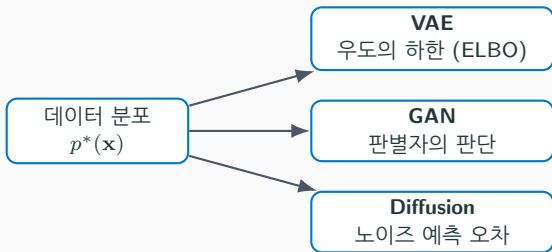
조사할 때의 원칙

생성형 AI에게 “답”을 묻지 말고, “어떤 논문의 어느 부분을 읽어야 하는지”를 물으십시오.

Recap: 세 갈래의 생성 모델

생성 모델을 나누는 기준

- 목표는 셋 다 같습니다: 데이터 분포 $p^*(\mathbf{x})$ 를 모사해 새로운 $\mathbf{x}$ 를 만드는 것.
- 차이는 “무엇을 학습 신호로 삼는가” 하나입니다.



	VAE	GAN	Diffusion
샘플링 비용	신경망 1회	신경망 1회	신경망 T 회
학습 안정성	안정적	불안정	안정적
전형적 약점	흐릿한 샘플	mode collapse	느린 생성

지난 주 Research Question

VAE는 왜 흐릿한(blurry) 샘플을 생성하는가?

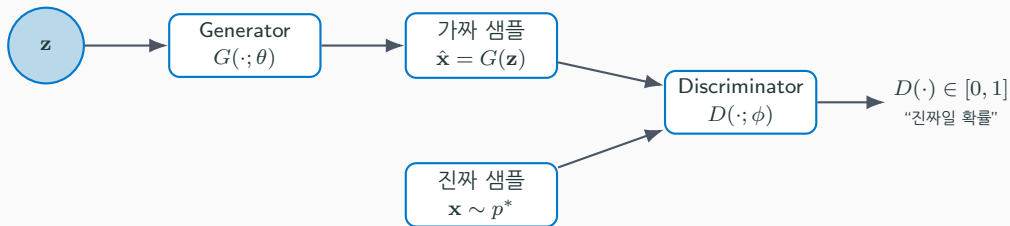
복습

- 디코더를 가우시안으로 가정하면 reconstruction loss가 **MSE**가 됩니다.
- MSE를 최소화하는 예측값은 **조건부 평균** $\mathbb{E}[\mathbf{x} \mid \mathbf{z}]$ 입니다.
- 하나의 $\mathbf{z}$ 에 대응할 수 있는 $\mathbf{x}$ 가 여러이면 모델은 그 후보들의 **평균**을 출력합니다 $\Rightarrow$ 흐릿함.

그렇다면: GAN과 Diffusion은 이 “평균화”를 각각 어떻게 피하는가?

$\hookrightarrow$ GAN은 가정을 없애고, Diffusion은 문제를 쪼갭니다.

GAN



$$\min_{\theta} \max_{\phi} V(G, D) = \mathbb{E}_{\mathbf{x} \sim p^*} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log (1 - D(G(\mathbf{z})))] \quad (1)$$

- D 는 진짜와 가짜를 구분하도록, G 는 D 를 속이도록 학습됩니다.
- 데이터에 대한 **확률분포 가정이 없습니다** — 이것이 선명한 샘플의 원인입니다.
- GAN에 대한 배경은 이 한 장이 전부입니다. 나머지는 질문으로 다룹니다.

Research Question

GAN의 학습은 실제로 어떤 순서로 진행되는가? 무엇을 손실로 두고, 그 손실은 코드에서 어떻게 구현되는가?

생각해볼 점

- D 와 G 는 하나의 손실을 함께 줄이는가, 아니면 각자 다른 손실을 줄이는가?
- “진짜/가짜를 맞추는 문제”는 결국 어떤 익숙한 문제인가?
- G 를 업데이트할 때 D 는 어떤 상태여야 하는가?

Q. GAN의 학습 순서와 손실, 그리고 그 구현은?

- 하나의 손실이 아니라 두 개의 손실을 번갈아 줄입니다.
- 둘 다 결국 **이진 분류의 cross-entropy**입니다. D 는 진짜를 1, 가짜를 0으로 맞히고, G 는 **가짜를 1**이라고 우깁니다.
- G 의 손실로 $-\log D(G(\mathbf{z}))$ 를 쓰는 이유는, 원래 식 $\log(1 - D(G(\mathbf{z})))$ 가 학습 초기에 기울기가 거의 0이 되기 때문입니다.
- 구현의 핵심은 **기울기를 어느 쪽으로 흐르게 할 것인**가입니다.

```
bce = nn.BCEWithLogitsLoss()
```

```
# (1) D step : G는 고정
```

```
x_fake = G(torch.randn(n, z_dim))
```

```
loss_d = bce(D(x_real), ones) \
        + bce(D(x_fake.detach()), zeros)
```

```
opt_d.zero_grad(); loss_d.backward(); opt_d.step()
```

```
# (2) G step : 라벨만 1로 바꿔 끼움
```

```
loss_g = bce(D(x_fake), ones)
```

```
opt_g.zero_grad(); loss_g.backward(); opt_g.step()
```

읽는 법: detach()는 D 를 학습할 때 기울기가 G 로 흘러가지 않게 끊는 것. 두 번째 단계에서 ones를 쓰는 것이 곧 $-\log D(G(\mathbf{z}))$ 입니다.

Research Question

GAN의 대표적 실패로 **mode collapse**가 꼽힌다. 이것은 어떤 현상이며, 왜 발생하는가?

생각해볼 점

- 데이터에 봉우리가 8개인데 생성기가 그중 하나만 완벽하게 만든다면, G 의 손실은 큰가 작은가?
- 목적함수 어디에 “모든 봉우리를 덮어라”는 요구가 있는가?
- 판별자가 그 한 봉우리를 알아채면 생성기는 다음에 무엇을 하겠는가?

Q. *Mode collapse*는 어떤 현상이며 왜 발생하는가?

간단한 답

- **현상:** 생성기가 데이터의 일부(때로는 단 하나)의 붐우리만 반복해서 만듭니다. 서로 다른 z 를 넣어도 거의 같은 샘플이 나옵니다.
- **원인 1 — 목적함수:** G 의 손실은 “판별자를 속여라”만 요구합니다. 진짜 같은 샘플 한 종류만 만들어도 손실은 충분히 낮아집니다. **다 덮으라는 향이 애초에 없습니다.**
- **원인 2 — 게임의 동역학:** 판별자가 그 붐우리를 잡아내면 생성기는 다른 붐우리로 통째로 옮겨갑니다. 넓혀가는 것이 아니라 **옮겨 다닙니다.**
- VAE와 정반대입니다. VAE는 다 덮으려다 뭉개고, GAN은 선명한 대신 빠뜨립니다.

Research Question

정형데이터 생성기에 mode collapse가 일어났다. 그 합성데이터를 받아 쓰는 사람에게는 어떤 문제가 생기는가?
그리고 그 사실을 어떻게 알아챌 수 있는가?

생각해볼 점

- 열별 평균과 분산만 비교해서 collapse를 발견할 수 있는가?
- 전체의 1%뿐인 희귀 범주가 사라지면 어떤 분석이 먼저 깨지는가?

직접 확인해보기: 2차원 8 Gaussians에서 GAN을 학습시키고, 생성 샘플이 몇 개의 봉우리를 덮는지
산점도로 세어 보십시오.

Q. Mode collapse는 합성데이터에 어떤 문제를 일으키며 어떻게 진단하는가?

간단한 답

- **희귀 범주와 꼬리의 소실**: 소수 집단, 이상치, 드문 조합이 통째로 사라집니다. 그런데 평균과 분산은 얼추 맞을 수 있어 **요약 통계만 보면 발견되지 않습니다**.
- **좋아 보이는 착시**: 남은 샘플은 진짜 같으므로 “얼마나 닮았는가”는 높게 나옵니다. 떨어지는 것은 **얼마나 덮었는가(coverage)**입니다. 두 축을 함께 봐야 합니다.
- **상관 구조의 왜곡**: 특정 조합만 반복되면 열 사이의 관계가 실제와 달라집니다. 열 하나씩 볼 때는 통과하는데 **두 열을 같이 보면 깨지는 패턴**이 전형적입니다.
- **다운스트림 성능**: 합성데이터로 학습한 모델이 소수 집단에서만 실패합니다. 단순한 성능 저하가 아니라 **공정성 문제**가 됩니다.
- **진단**: 범주별 빈도표 비교, 생성 샘플 안의 중복 비율, 두 열씩 짝지은 산점도, 2차원 투영(PCA) 시각화.

Diffusion: 최소한의 배경

Forward process (학습 대상이 아님)



$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t | \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad \beta_1, \dots, \beta_T : \text{fixed schedule}$$

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t | \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s) \quad (2)$$

- forward는 학습하지 않습니다. 정해진 규칙대로 노이즈를 더할 뿐입니다.
- 그래서 임의의 t 에 대해 한 번에 $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ 를 만들 수 있습니다 ($\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$).
- 주의: 여기서의 β_t 는 Week 2의 β -VAE의 β 와 무관합니다.

- 역방향 $p(\mathbf{x}_{t-1} | \mathbf{x}_t; \theta)$ 를 가우시안으로 두고 그 평균을 학습합니다. 실제 구현에서는 평균 대신 **더해진 노이즈**를 예측합니다.

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|_2^2 \right], \quad t \sim \mathcal{U}\{1, \dots, T\} \quad (3)$$

학습 (한 스텝)

1. $\mathbf{x}_0, t, \epsilon$ 샘플링
2. $\mathbf{x}_t$ 를 위 식으로 한 번에 계산
3. $\|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2$ 로 역전파

생성 (샘플링)

1. $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2. $t = T, \dots, 1$ 에 대해 조금씩 노이즈 제거
3. $\mathbf{x}_0$ 출력 (신경망 T 회 호출)

학습은 **단순한 회귀 문제**가 되었고, 어려움은 전부 **샘플링 비용**으로 옮겨갔습니다.

Research Questions — Diffusion

Research Question

보통 $T = 1000$ 으로 둔다. 그러면 denoising 단계도 1000개다. 그런데 우리가 학습하는 신경망은 **1000개가 아니라 하나뿐**이다. 어떻게 가능한가? 그 신경망은 **지금 몇 번째 단계인지**를 어떻게 아는가?

생각해볼 점

- ϵ_θ 의 입력은 몇 개인가? t 는 파라미터인가, 입력인가?
- $t = 500$ 에서의 denoising과 $t = 501$ 에서의 denoising은 얼마나 다른 문제인가?
- 1000개를 따로 둔다면 각 신경망은 데이터를 얼마나 보게 되는가?

Q. $T = 1000$ 인데 신경망이 하나인 것이 어떻게 가능한가?

간단한 답

- 신경망은 $\epsilon_{\theta}(\mathbf{x}_t, t)$ 입니다. t 는 **파라미터가 아니라 입력**이고, 하나의 신경망이 모든 단계를 **공유**합니다. 단계마다 달라지는 것은 가중치가 아니라 **입력으로 들어가는 조건**입니다.
- 구현: t 를 벡터로 바꿔(**timestep embedding**) 신경망에 함께 넣습니다. 이것이 빠지면 모델은 노이즈가 얼마나 섞였는지 모른 채 “평균적인 denoiser”가 되어 학습이 무너집니다.
- 가능한 이유: $t = 500$ 과 $t = 501$ 의 문제는 거의 같습니다. 1000개의 다른 함수가 아니라, **노이즈 수준에 따라 조금씩 변하는 하나의 함수**이므로 그것만 배우면 됩니다.
- 오히려 공유가 유리합니다. 따로 두면 각 신경망이 데이터의 1/1000만 보게 되지만, 공유하면 **모든 단계의 데이터가 서로를 돕습니다**.

Research Question

노이즈를 주입하는 스케줄에 따라 생성된 표본이 달라진다. **linear, cosine, quadratic** 스케줄은 무엇이 다른가?

생각해볼 점

- 스케줄이 직접 정하는 것은 β_t 인데, 실제로 중요한 것은 누적된 $\bar{\alpha}_t$ 입니다. 왜인가?
- 마지막 몇 백 스텝이 이미 순수 노이즈라면, 그 스텝들은 무슨 일을 하고 있는가?

직접 확인해보기: 세 스케줄의 $\bar{\alpha}_t$ 곡선을 한 그림에 겹쳐 그리고, 같은 $\mathbf{x}_0$ 에서 $t = 0, T/4, T/2, 3T/4, T$ 의 $\mathbf{x}_t$ 를 나란히 시각화해 보십시오.

Q. *linear / cosine / quadratic* 스케줄은 무엇이 다른가?

간단한 답

- 스케줄이 정하는 것은 결국 하나, **정보가 사라지는 속도**입니다. 그래서 봐야 할 것은 β_t 가 아니라 누적된 $\bar{\alpha}_t$ 곡선입니다.
- **linear**: 가장 단순하지만 후반 구간이 이미 거의 순수 노이즈여서 그 스텝들이 사실상 **낭비**됩니다.
- **cosine**: 중간 구간에서 완만히 감소하도록 설계해 **쓸모 있는 스텝이 늘어납니다**. 대체로 품질이 개선됩니다.
- **quadratic**: 초반 정보를 더 오래 보존하고 뒤에서 몰아 파괴합니다. 구현체마다 정의가 다르니 반드시 곡선을 직접 그려 확인하십시오.
- **반드시 점검할 두 끝점**: $\bar{\alpha}_T \approx 0$ 이어야 샘플링 시작점 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 와 학습이 일치하고, $\bar{\alpha}_1 \approx 1$ 이어야 복원이 됩니다.
- **정형데이터 주의**: 이미지보다 정보량이 적어 훨씬 빨리 파괴됩니다. 이미지용 스케줄과 T 를 그대로 쓰면 대부분의 스텝이 의미 없이 소모됩니다.

Research Question

ϵ 을 예측하는 것과 $\mathbf{x}_0$ 을 예측하는 것은 서로 옮길 수 있어 **수식적으로 동일**하다. 그런데 실제 구현은 거의 항상 ϵ 예측이다. 왜 그런가?

생각해볼 점

- t 가 매우 클 때, 즉 거의 순수 노이즈일 때 $\mathbf{x}_0$ 를 맞추는 것이 가능한 일인가?
- 두 경우에 **맞혀야 할 값의 크기**는 t 에 따라 어떻게 변하는가?

직접 확인해보기: 같은 데이터 · 같은 구조로 ϵ 예측과 $\mathbf{x}_0$ 예측을 각각 학습시키고, t 별 손실과 최종 샘플을 비교해 보십시오.

Q. 수식적으로 동일한데 왜 ϵ 예측으로 구현하는가?

간단한 답

- 서로 옮길 수는 있지만, MSE를 취하는 순간 t 마다 다른 가중치가 붙습니다. 두 손실은 $\|\mathbf{x}_0 - \hat{\mathbf{x}}_0\|^2 = \frac{1-\bar{\alpha}_t}{\bar{\alpha}_t} \|\epsilon - \hat{\epsilon}\|^2$ 로 연결되고, 이 계수는 t 에 따라 크게 달라집니다. **무엇을 예측하느냐 = 어느 t 를 중요하게 학습하느냐**입니다.
- $\mathbf{x}_0$ 예측은 노이즈가 큰 구간에 가중치 몰립니다. 그런데 그 구간에서는 원본을 알아낼 방법이 없으므로, 모델은 데이터 평균만 배우고 학습 신호는 대부분 잡음이 됩니다.
- ϵ 은 t 와 무관하게 항상 표준정규분포에서 나온 값입니다. **맞혀야 할 값의 크기가 일정**하므로 학습이 안정적입니다.
- **만능은 아닙니다.** T 가 작은 정형데이터에서는 $\mathbf{x}_0$ 예측이 더 나은 사례도 있습니다. “원래 그렇게 한다”가 아니라 **직접 비교해** 보고 정하십시오.

Research Question

DDPM의 실제 손실은 모든 t 의 항에 **같은 가중치**를 준다. 그러나 정확한 ELBO에서는 t 가 아주 작을 때의 항이 훨씬 **큰 가중치**를 갖는다. 그 항을 사실상 무시하는 이유는? 무시하면 안 되는 경우는?

생각해볼 점

- $t = 1$ 일 때 $\mathbf{x}_1$ 은 $\mathbf{x}_0$ 와 얼마나 다른가? 그 단계에서 모델이 할 일은 무엇인가?
- 전체적인 모양(어떤 사람인지, 어떤 고객 유형인지)은 큰 t 와 작은 t 중 어디서 결정되는가?

Q. 작은 t 의 denoising 항을 왜 사실상 무시하는가?

간단한 답

- 정확한 ELBO에서는 작은 t 항일수록 큰 가중치가 붙습니다. DDPM은 그 가중치를 **모두 1로 바꿔버립니다**. 결과적으로 작은 t 를 **덜 중요하게** 취급하는 셈입니다.
- 이유: $t \approx 1$ 에서는 $\mathbf{x}_t$ 가 이미 $\mathbf{x}_0$ 와 거의 같아서, 남은 일은 **아주 미세한 마무리**뿐입니다. 거기에 큰 가중치를 주면 모델이 눈에 띄지도 않는 디테일에 매달리고, 전체 모양을 결정하는 **큰 t 의 어려운 문제**를 소홀히 하게 됩니다.
- 즉 “가중치를 버린 것”은 실수가 아니라, **샘플 품질을 위한 의도된 선택**입니다. 실제로 이렇게 두었을 때 샘플이 더 좋아진다는 것이 DDPM의 실험 결과입니다.
- **무시하면 안 되는 경우**: 로그우도를 정확히 평가해야 하거나, 정형데이터에서 각 열의 값을 정확히 복원해야 할 때는 바로 그 작은 t 구간이 그 일을 담당합니다.

Research Question

1000 스텝으로 학습한 모델이 있다. 이 모델을 1000번 denoising하지 않고 **50번 또는 100번**만으로 샘플링할 수 있는가? 가능하다면 왜 가능하며, 품질은 어떻게 되는가?

생각해볼 점

- 학습으로 얻은 것은 “1000단계짜리 절차” 인가, 아니면 함수 $\epsilon_{\theta}(\mathbf{x}_t, t)$ 인가?
- t 를 1000, 980, ...처럼 건너뛰며 넣어도 되는 이유는?

직접 확인해보기: 같은 모델로 스텝 수를 1000, 200, 100, 50, 20으로 바꿔 샘플링하고, 어디서부터 무너지는지를 확인해 보십시오.

Q. 1000 스텝으로 학습한 모델을 50-100 스텝으로 샘플링할 수 있는가?

간단한 답

- **가능합니다.** 학습으로 얻은 것은 절차가 아니라 함수 $\epsilon_{\theta}(\mathbf{x}_t, t)$ 이고, 이 함수는 어떤 노이즈 수준에서든 답을 줍니다. 그래서 샘플링할 때 t 를 건너뛰며 성글게 잡아도 그대로 쓸 수 있습니다.
- 다만 원래의 DDPM 샘플러를 그냥 성글게 쓰면, 매 스텝 다시 넣는 무작위성 때문에 오차가 쌓여 빠르게 나빠집니다.
- **DDIM**은 그 무작위성을 없앤 샘플러입니다. **같은 모델을 다시 학습하지 않고** 그대로 쓰면서, 50 스텝 정도까지는 품질 저하가 크지 않습니다. 20 아래로 내려가면 눈에 띄게 무너집니다.
- **잃는 것:** 세밀한 디테일과 약간의 다양성. **얻는 것:** 10-20배의 속도.
- 정형데이터는 차원이 낮아 50-100 스텝으로 충분한 경우가 많습니다. 스텝 수를 바꿔가며 직접 재보고 정하는 것이 맞습니다.

Research Question

Diffusion은 **연속적인** 가우시안 노이즈에 기반한다. 나이나 소득 같은 수치형 열은 그렇다 치더라도, **성별이나 직업 같은 범주형 열**에는 어떻게 노이즈를 더할 것인가?

생각해볼 점

- one-hot 벡터에 가우시안 노이즈를 더하면 (0.3, 0.4, 0.3) 같은 값이 나온다. 이것은 무엇인가?
- 범주형에서 “노이즈를 점점 더한다”를 정의한다면, 최종 상태는 무엇이어야 하는가?

직접 확인해보기: 범주형 열을 one-hot으로 두고 가우시안 diffusion을 돌린 뒤 $\arg \max$ 로 되돌렸을 때, 원본과 **범주 빈도**가 얼마나 어긋나는지 보십시오.

Q. 정형데이터, 특히 범주형 열에 Diffusion을 어떻게 적용하는가?

간단한 답

- **연속형**: Week 2의 전처리(quantile / Gaussian transform)를 그대로 쓴 뒤 표준적인 가우시안 diffusion을 적용하고, 생성 후 역변환합니다.
- **범주형** — 두 가지 길: **(a)** one-hot을 그냥 실수로 보고 노이즈를 더한 뒤 $\arg \max$ 로 되돌리기.
간단하지만 회귀 범주가 특히 왜곡됩니다. **(b) multinomial diffusion** — 매 스텝 일정 확률로 범주를 무작위로 바꿔 최종 상태가 **균등분포**가 되도록 정의하기.
- **TabDDPM**은 (b)를 택해, 연속형은 가우시안 · 범주형은 multinomial로 두고 두 손실을 더해 하나의 모델로 함께 학습합니다.
- **이미지와 달라야 하는 것**: 차원이 낮으므로 T 가 훨씬 작아도 되고, U-Net 대신 **MLP**로 충분한 경우가 많습니다. 스케줄도 다시 잡아야 합니다 (RQ 2).
- **모델이 지켜주지 않는 것**: 나이 ≥ 0 , 합계 일치, 퇴사일 $\geq$ 입사일 같은 도메인 제약과 결측 패턴은 따로 후처리해야 합니다.

Research Questions — 세 모델 공통

공통 RQ 1. 세 가지 실패는 서로 무관한가

Research Question

VAE는 흐릿하고, GAN은 **mode**를 빠뜨리고, Diffusion은 느리다. 이 세 가지 실패는 각 모델의 우연한 결함인가, 아니면 하나의 원리에서 나오는 서로 다른 얼굴인가?

생각해볼 점

- VAE는 다 덮으려다 무엇을 잃었고, GAN은 선명함을 얻으려다 무엇을 잃었는가?
- Diffusion은 VAE처럼 다 덮는데 왜 흐릿하지 않은가? 대신 무엇을 지불했는가?

Q. *blur*, *mode collapse*, 느린 생성은 하나의 원리로 설명되는가?

간단한 답

- 결국 한 가지 질문으로 정리됩니다: “모델에게 한 번에 얼마나 어려운 일을 시키는가?”
- **VAE**: 데이터를 다 덮는 방향으로 학습하지만, z 하나에서 x 전체를 한 번에 만들어야 합니다. 후보가 여럿이면 평균을 출력 $\Rightarrow$ 덮되 뭉개다.
- **GAN**: 판별자만 속이면 되므로 한 봉우리에 몰려도 별점이 없습니다. 분포 가정이 없어 평균화도 없습니다 $\Rightarrow$ 선명하되 빠뜨린다.
- **Diffusion**: VAE처럼 다 덮되, 생성을 T 단계로 쪼개 각 단계에서 조금만 하게 했습니다. 한 번에 하는 일이 쉬우면 평균을 내도 뭉개지지 않습니다 $\Rightarrow$ 덮으면서 선명, 대신 계산을 T 배 지불.
- 즉 세 실패는 (덮기 — 뭉개기 — 계산량)이라는 하나의 삼각형 위의 서로 다른 지점입니다.

Research Question

VAE, GAN, Diffusion으로 각각 합성 정형데이터를 만들었다. 무엇이 가장 좋은지를 어떻게 판정할 것인가? 원본과 통계적으로 완벽히 같은 합성데이터는 최고의 결과인가?

생각해볼 점

- 원본을 그대로 복사해서 제출하면 각 지표에서 몇 점을 받는가?
- 세 모델을 공정하게 비교하려면 무엇을 똑같이 맞춰야 하는가?

Q. 좋은 합성데이터의 기준은 무엇이며, 완벽한 일치는 최고의 결과인가?

간단한 답

- 최소 네 가지를 함께 봐야 합니다: **닮았는가(fidelity)**, **다 덮었는가(coverage)**, **쓸모 있는가(utility)**, **원본을 흘리지 않는가(privacy)**.
- 정형데이터의 기본 지표: 열별 분포 비교, 열 사이 상관 행렬의 차이, **TSTR**(합성으로 학습 → 실제로 평가), 그리고 원본과 지나치게 가까운 행의 비율.
- 두 번째 질문의 답은 **아니오**입니다. 원본을 그대로 복사하면 “**닮았는가**”는 만점이지만 **privacy**는 0 점입니다. **fidelity**만으로 **고르면 가장 많이 외운 모델이 항상 이깁니다**.
- **공정 비교의 조건**: 같은 전처리, 같은 생성 샘플 수, 같은 평가 코드, 여러 시드의 평균. 하나라도 다른 모델 비교가 아니라 파이프라인 비교가 됩니다.

정리

같은 목표를 두고 서로 다른 것을 포기한 세 가지 방법

- **GAN**은 분포 가정을 없애서 선명함을 얻고, 그 대가로 **다양성**을 잃었습니다.
- **Diffusion**은 문제를 여러 개의 쉬운 문제로 쪼개서 안정성과 다양성을 얻고, 그 대가로 **속도**를 잃었습니다.
- **VAE**는 다 덮는 대신 **선명함**을 내주었습니다.
- 합성 정형데이터에서는 여기에 **privacy**와 **제약 만족**이 더해집니다.

스스로 해볼 것

- ☐ 오늘의 11개 질문 중 **3개 이상**을 골라 A4 반 페이지씩 정리
- ☐ 고른 질문의 “찾아볼 키워드”로 관련 논문 1편의 해당 절 읽기
- ☐ “직접 확인해보기”가 붙은 질문 중 하나는 코드로 확인

GAN

- I. Goodfellow et al., "Generative Adversarial Nets," NeurIPS, 2014.
- Z. Lin et al., "PacGAN: The Power of Two Samples in Generative Adversarial Networks," NeurIPS, 2018.
- L. Xu et al., "Modeling Tabular Data using Conditional GAN," NeurIPS, 2019.

Diffusion

- J. Ho et al., "Denoising Diffusion Probabilistic Models," NeurIPS, 2020. (ϵ 예측, $\mathcal{L}_{\text{simple}}$)
- J. Song et al., "Denoising Diffusion Implicit Models," ICLR, 2021. (샘플링 가속)
- A. Nichol and P. Dhariwal, "Improved DDPM," ICML, 2021. (cosine schedule)
- A. Kotelnikov et al., "TabDDPM: Modelling Tabular Data with Diffusion Models," ICML, 2023.

평가 / 합성데이터

- M. Sajjadi et al., "Assessing Generative Models via Precision and Recall," NeurIPS, 2018.
- N. Patki et al., "The Synthetic Data Vault," IEEE DSAA, 2016.